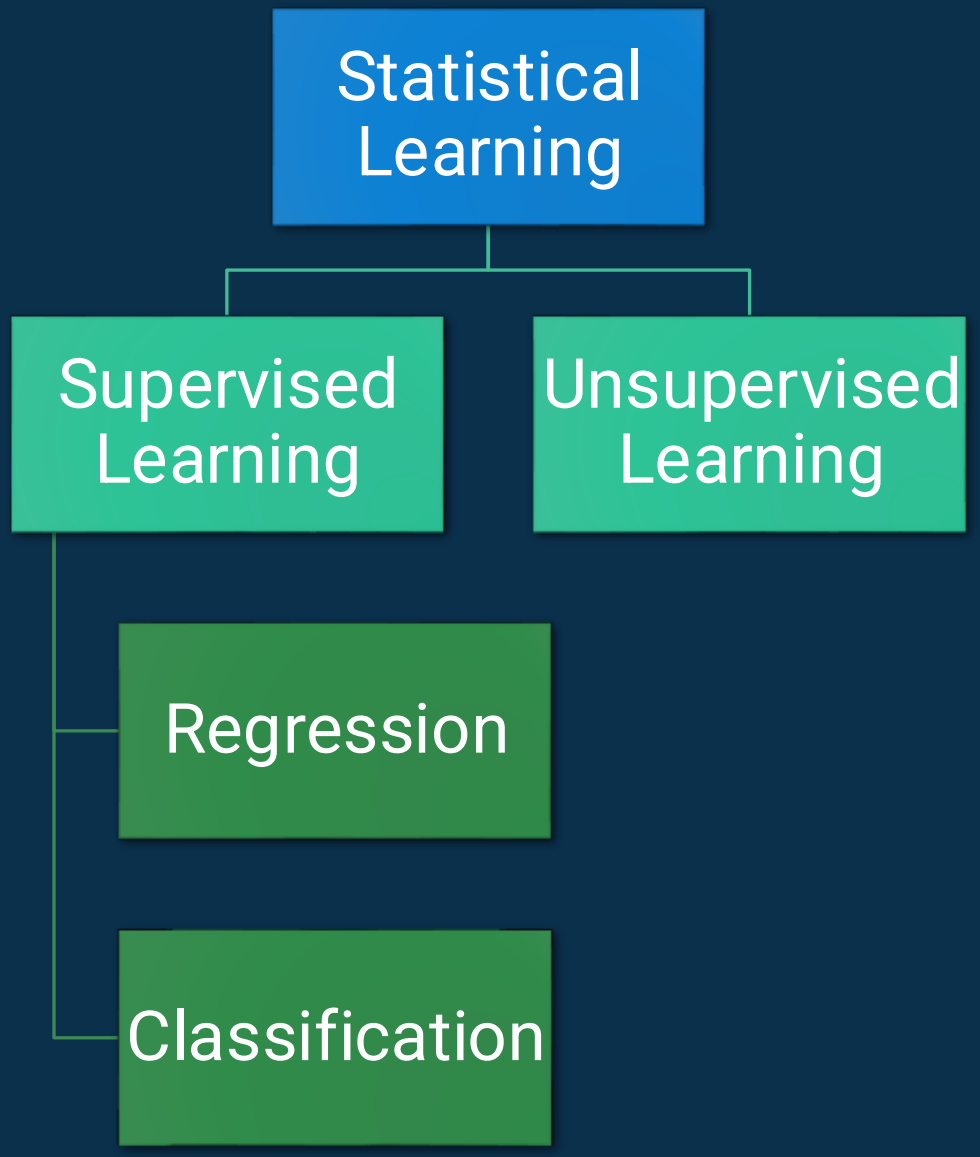
Session 4 (DAY 4)

Case Study

Workshop on Quantitative Literacy and Statistics

UNIVERSITY OF NEBRASKA AT OMAHA
DATA AND DECISION SCIENCES


Statistical Learning
Supervised Learning
Unsupervised Learning
Regression
Classification

Supervised Learning: Regression

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis (EDA)
- Descriptive Statistics
- Plot the data
- Find correlations

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

Workshop on Quantitative Literacy and Statistics: Session 4 - Case Study

Important: Before Start

1. **Copy Excel File `sample_data_1.xlsx` to a known folder in your computer**
2. **Open R Studio and create a new R file in the folder where you copied the Excel File**
3. **In R Studio: Click `Session` tab** → Click `Set Working Directory` → Click `Source File Location`

Required Packages

```
suppressMessages({
if(!require(readxl)){install.packages("readxl")}
if(!require(corrplot)){install.packages("corrplot")}
if(!require(knitr)){install.packages("knitr")}
if(!require(ggplot2)){install.packages("ggplot2")}
if(!require(rpart)){install.packages("rpart")}
if(!require(rpart.plot)){install.packages("rpart.plot")}
if(!require(randomForest)){install.packages("randomForest")}
if(!require(caret)){install.packages("caret")}
if(!require(factoextra)){install.packages("factoextra")}
if(!require(FactoMineR)){install.packages("FactoMineR")}

library("readxl")
library(corrplot)
library(knitr)
library(ggplot2)
library(rpart)
library(rpart.plot)
library(randomForest)
library(caret)
library(factoextra)
library(FactoMineR)
})
```

1 Brief Description of the Data Set:

`Mouse Model of Glaucoma`

Data Provide By: Thompson, Jenny L <jennthompson@unmc.edu>

We use a mouse model of glaucoma. We're seeking to uncover the influence of intraocular pressure (IOP) on different variables in a bilateral brain region that receives input from both eyes. The data are obtained from tissue that contains input from BOTH eyes, then run the computations with data from the LEFT eye and RIGHT eye separately.

Independent Variables:

- AGE is categorized (4m, 9m, 12m) with groups relatively balanced, but I have exact values, so can be continuous
- STRAIN is categorical (D2s get glaucoma, D2-Controls = same genetic background sans glaucoma mutation)

Dependent Variables:

- IOP (intraocular pressure) is recorded in mmHg and is measured in each eye separately; these are taken monthly
- IOP AUC I use the area under the curve for IOP so that I can maximize the use of the longitudinal data
- PUNCTA DENSITY is a value telling the density of area covered by a fluorescent signal
- BRANCH LENGTH is the average branching of microglial cells in the tissue
- ENDPOINTS average number of endpoints of microglia in the tissue; derived from same output as
- BRANCH LENGTH; these two variables do strongly correlate and I believe they should be combined into one variable (i.e. a "ramification factor"), but they do indicate different features, so I am in need of advice on this please
- C1Q INTENSITY is the peak intensity of a fluorescent signal in the tissue
- MICROGLIA COUNT number of microglia in 3d volume of tissue; manual counts

Data Inspection and Preprocess

Supervised Learning: Regression

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis
- Descriptive Statistics
- Plot the data

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

2 Data Inspection

2.1 Read Data Set: `Mouse Model of Glaucoma`

Load data set from the excel file named `sample_data_1.xlsx`. Here we create two data sets for experiment (or test) and control.

```
library("readxl")

glaucoma_experiment    = as.data.frame(read_excel("Data Sets/sample_data_1.xlsx", sheet = "D2"))
glaucoma_control       = as.data.frame(read_excel("Data Sets/sample_data_1.xlsx", sheet = "D2 Contro
l"))
```

Let's observe the first few observations of the experiment (or test) data where 'Strain' is `D2/J`.

Let's observe the first few observations of the experiment (or test) data where 'Strain' is `D2/J`.

```
head(glaucoma_experiment)
```

Let's observe the first few observations of the control data where 'Strain' is `D2-C`.

```
head(glaucoma_control)
```

Let's observe the first few observations of the experiment (or test) data where 'Strain' is `D2/J`.

```
head(glaucoma_experiment)
```

```
##   Age Strain Sex Microglia vGlut2_Puncta_Density Endpoints IOP_Right IOP_Left AUC_R AUC_L
## 1   4   D2/J   F        23              7.870000  138.8696      13.5     14.5   525   483
## 2   4   D2/J   F        24              7.470000  128.7500      13.5     14.5   378   378
## 3   4   D2/J   M        25              6.930000  124.9600      13.0     11.0   399   441
## 4   4   D2/J   M        22              8.880000  145.8636      12.5     12.5   399   441
## 5   4   D2/J   M        18              8.151936  166.3889      12.5     11.5   420   420
## 6   4   D2/J   M        23              8.349160  269.6957       9.0      9.0   216   546
```

Let's observe the first few observations of the control data where 'Strain' is `D2-C`.

```
head(glaucoma_control)
```

```
##   Age Strain Sex Microglia vGlut2_Puncta_Density Endpoints IOP_Right IOP_Left AUC_R AUC_L
## 1   4   D2-C   F        26                 7.867   167.200       9.5     10.5   266   294
## 2   4   D2-C   M        24                 7.728   131.100      10.5     11.0   294   308
## 3   4   D2-C   F        22                 9.795   150.820      11.5     12.5   322   350
## 4   4   D2-C   M        21                 7.867   145.000      12.5     14.0   350   392
## 5   4   D2-C   M        18                 7.809   158.222      13.0     14.0   364   392
## 6   4   D2-C   M        14                 7.743   202.200      11.5     10.5   322   294
```

2.2 Merge Experiment and Control Data Set

For further analysis of data we combined the experiment data set and test data set using `rbind` function in `R`.

```
glaucoma_df = rbind(glaucoma_experiment, glaucoma_control)
```

Lets make another quick look at the data set and check the column names and data set size.

```
# First five observations
head(glaucoma_df, 5)
```

Experiment Data

Control Data

```r
# First five observations
head(glaucoma_df, 5)
```

```
##   Age Strain Sex Microglia vGlut2_Puncta_Density Endpoints IOP_Right IOP_Left AUC_R AUC_L
## 1   4   D2/J   F        23              7.870000  138.8696      13.5     14.5   525   483
## 2   4   D2/J   F        24              7.470000  128.7500      13.5     14.5   378   378
## 3   4   D2/J   M        25              6.930000  124.9600      13.0     11.0   399   441
## 4   4   D2/J   M        22              8.880000  145.8636      12.5     12.5   399   441
## 5   4   D2/J   M        18              8.151936  166.3889      12.5     11.5   420   420
```

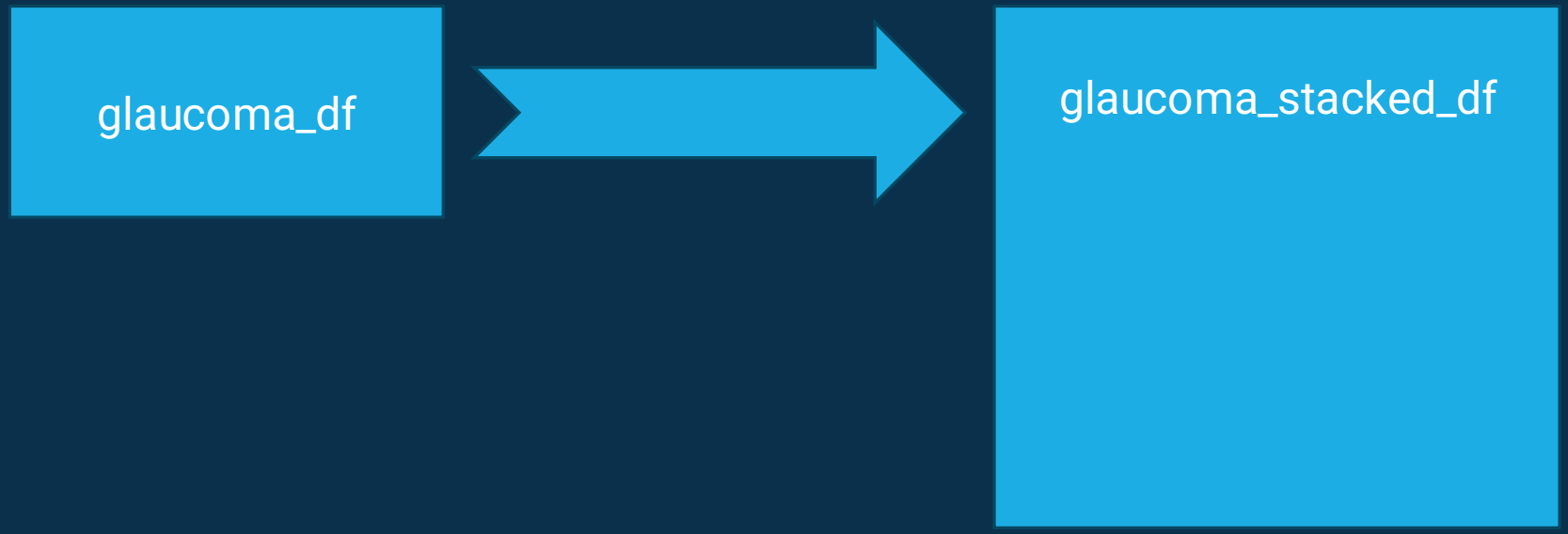
```r
# Last observations
tail(glaucoma_df,5)
```

```
##    Age Strain Sex Microglia vGlut2_Puncta_Density Endpoints IOP_Right IOP_Left AUC_R AUC_L
## 71  12   D2-C   M        25              7.041636 136.76000      13.0     12.0  3703  3339
## 72  12   D2-C   F        22              7.655000 164.18182      14.5     14.0  4070  4087
## 73  12   D2-C   F        24              7.600000 123.91667      16.0     16.5  4071  4045
## 74  12   D2-C   F        33              7.334000  98.30303      10.0      8.0  3812  3457
## 75  12   D2-C   F        28              5.763331 117.92857      14.5     15.5  3953  3996
```

2.3 Modify the Data Set by Stacking for Left and Right Eyes

For this data set, restructuring is beneficial for data visualization and modeling.

```
IOP = stack(glaucoma_df[7:8])
colnames(IOP) = c("IOP_Values","IOP_Eye")

#AUC = stack(glaucoma_df[9:10])
#colnames(AUC) = c("AUC_Values","AUC_Eye")

glaucoma_stacked_df = cbind(glaucoma_df[1:6],IOP)

head(glaucoma_stacked_df, n = 10)
```



```
##    Age Strain Sex Microglia vGlut2_Puncta_Density Endpoints IOP_Values   IOP_Eye
## 1    4   D2/J   F        23              7.870000  138.8696       13.5 IOP_Right
## 2    4   D2/J   F        24              7.470000  128.7500       13.5 IOP_Right
## 3    4   D2/J   M        25              6.930000  124.9600       13.0 IOP_Right
## 4    4   D2/J   M        22              8.880000  145.8636       12.5 IOP_Right
## 5    4   D2/J   M        18              8.151936  166.3889       12.5 IOP_Right
## 6    4   D2/J   M        23              8.349160  269.6957        9.0 IOP_Right
## 7    4   D2/J   M        23              7.669832  166.0435        9.5 IOP_Right
## 8    4   D2/J   M        32              7.881665   96.7500        9.5 IOP_Right
## 9    4   D2/J   M        23              7.406866  140.9565       10.0 IOP_Right
## 10   4   D2/J   M        23              8.086194  154.6087       12.0 IOP_Right
```

Exploratory Data Analysis

Supervised Learning: Regression

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis
- Descriptive Statistics
- Plot the data

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

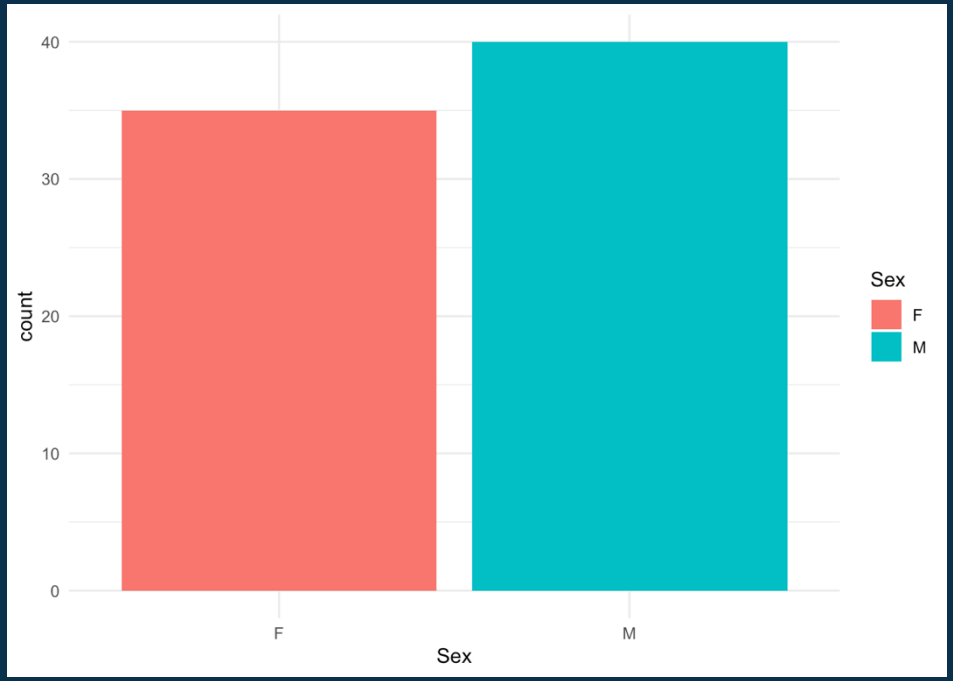
3 Data Visualization

In this section, we will apply different data visualization techniques for the variables in the data set.

3.1 Bar Charts for Categorical Variables

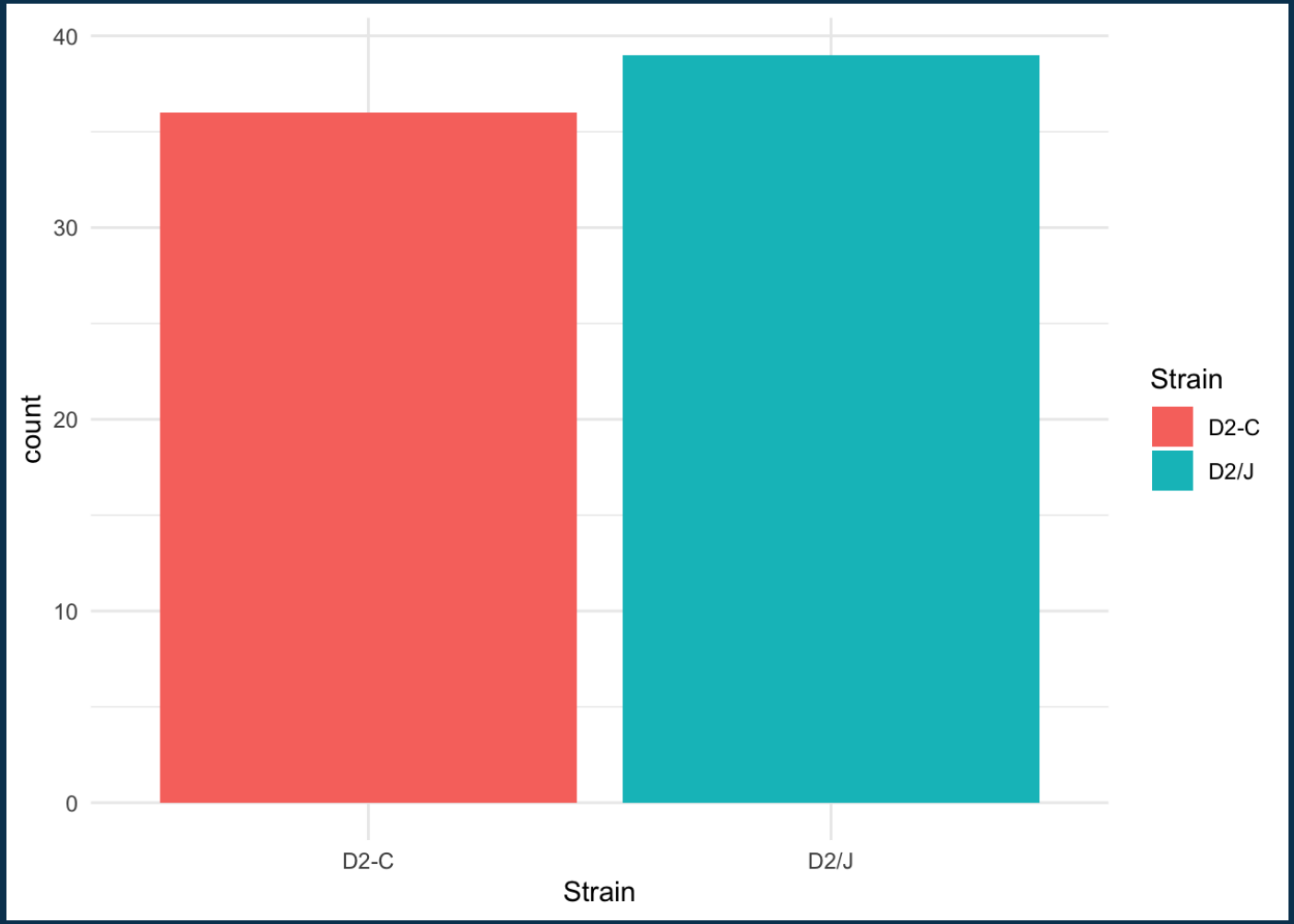
1. **Bar Cart for Sex Category Counts**

```
library(ggplot2)

ggplot(data=glaucoma_df, aes(x= Sex, fill = Sex)) +
  geom_bar(stat="count") + theme_minimal()
```



2. **Bar Cart for Strain Category Counts**

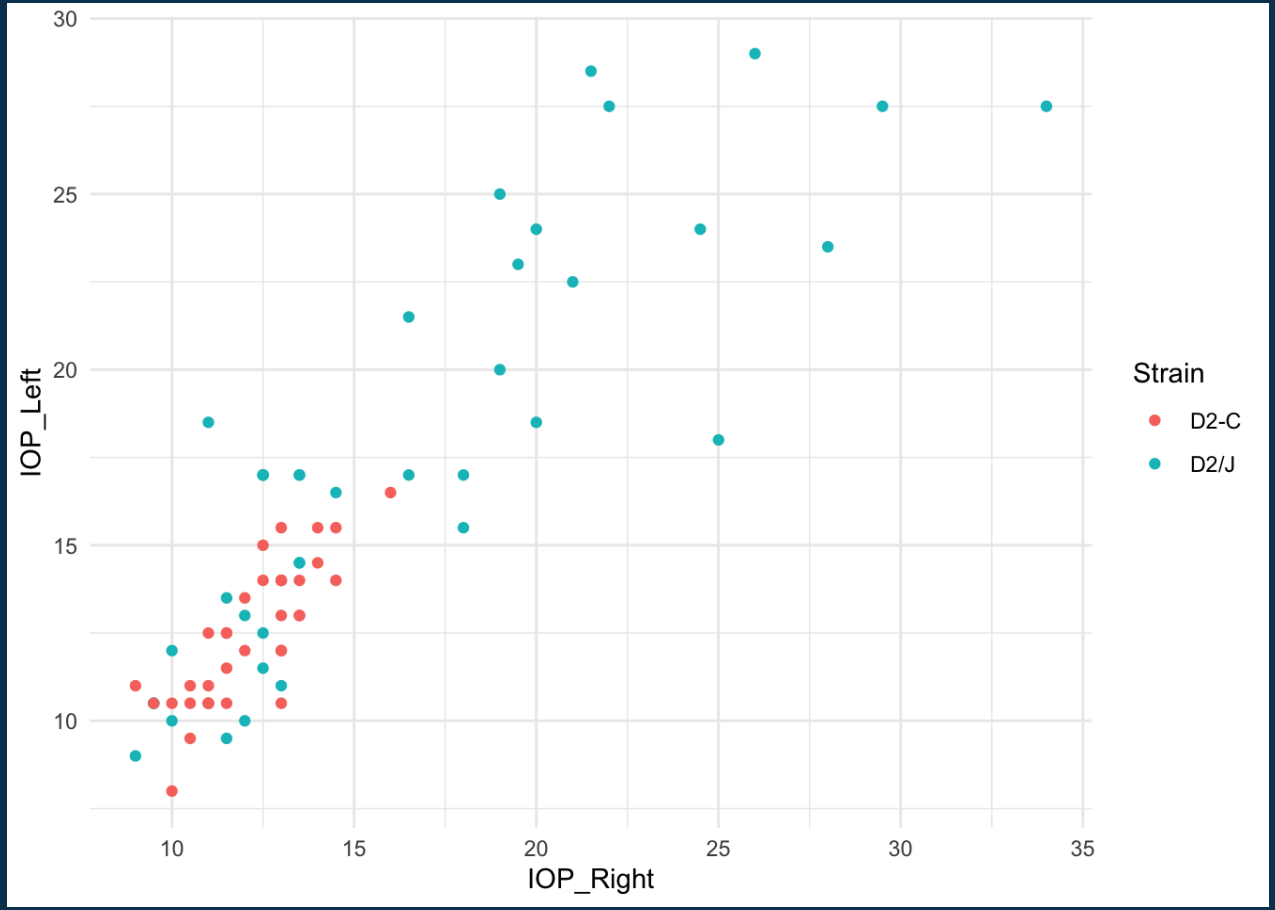
```
ggplot(data=glaucoma_df, aes(x= Strain, fill = Strain)) +
  geom_bar(stat="count") + theme_minimal()
```



3.2 Scatter Plot

A scatter plot is drawing for `IOP_Left` and `IOP_Right`. The points are colored based on the `Strain`.

```
ggplot(data=glaucoma_df, aes(x= IOP_Right, y = IOP_Left, color=Strain)) +
  geom_point() + theme_minimal()
```

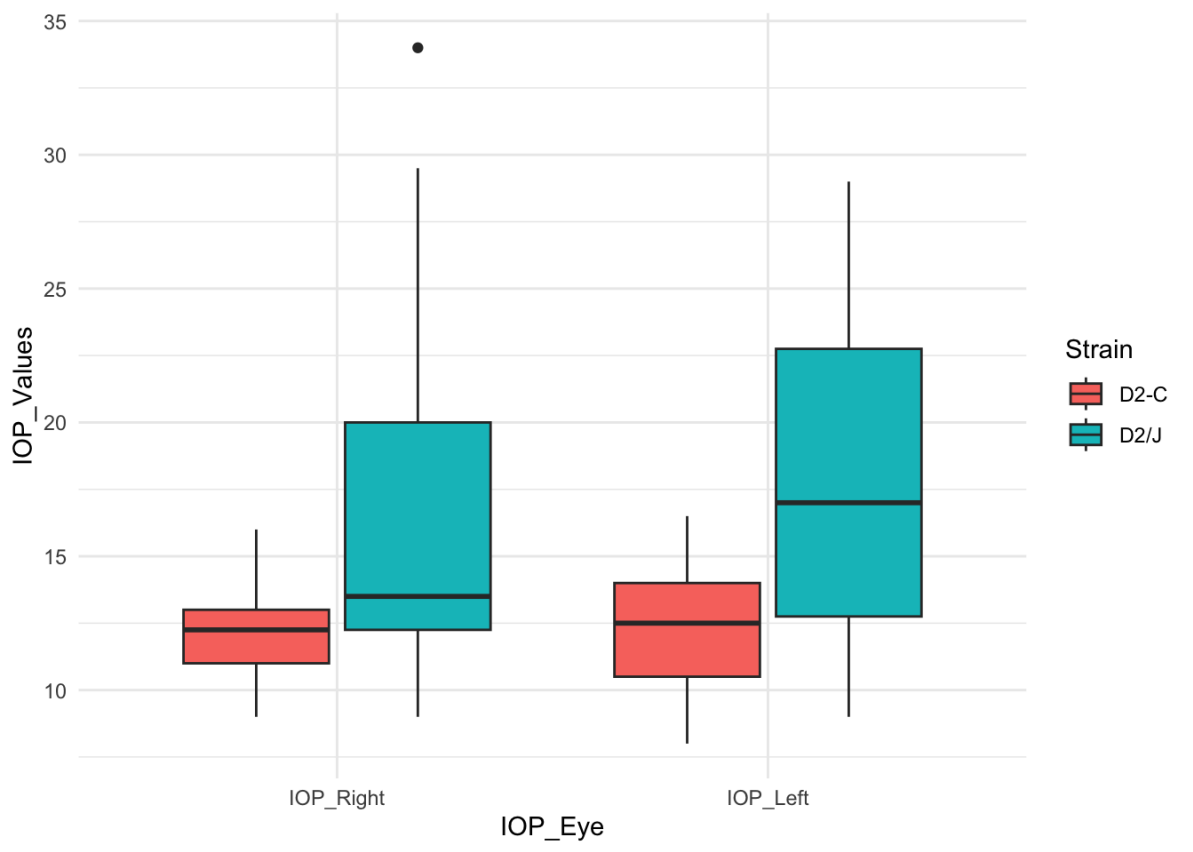


3.3 Box Plots

We draw box plots by groups, thereby, we can observe the data distribution and compare with other groups.

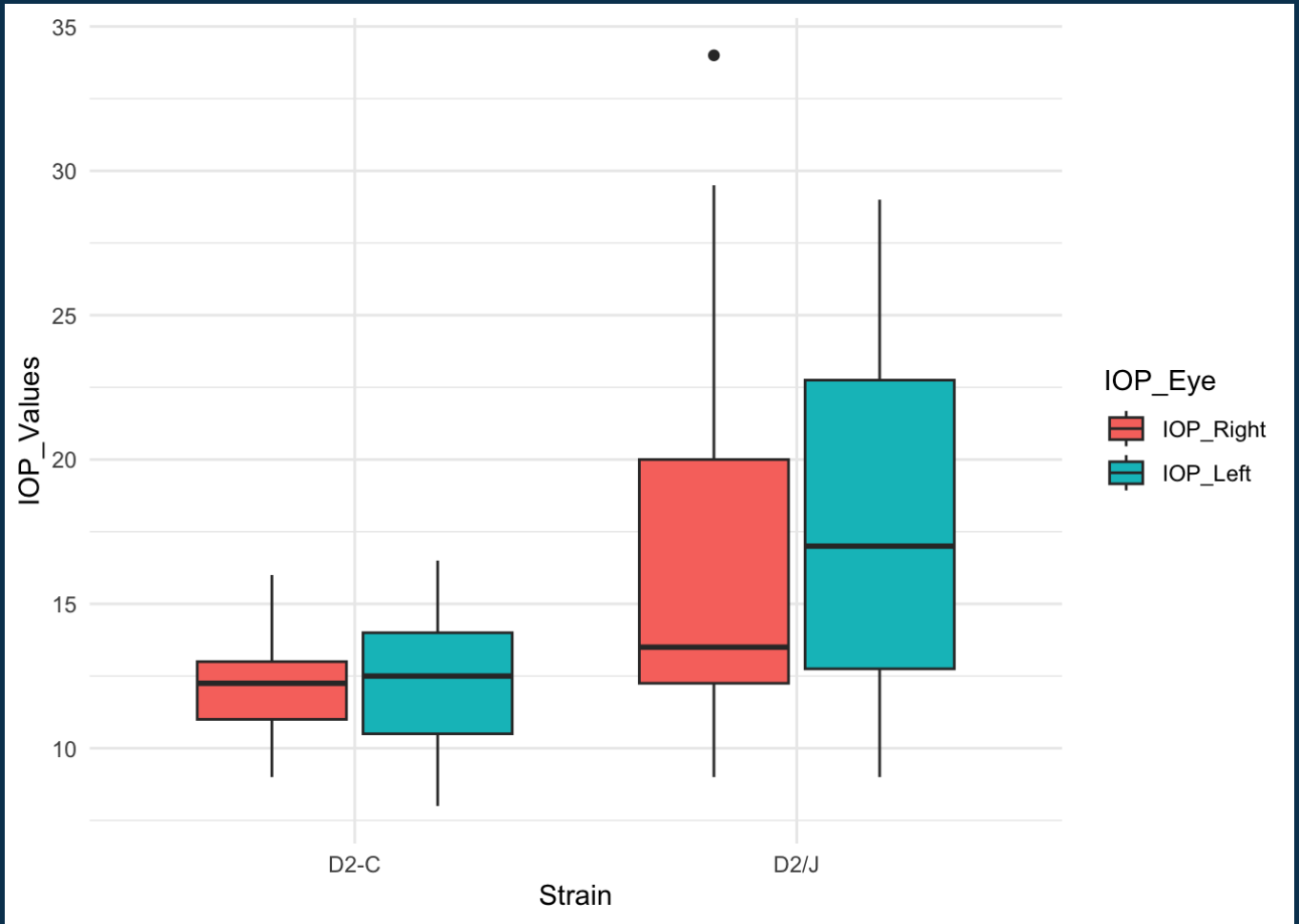
1. **Box plots for IOP values by Eye Side and by Strain**

```
ggplot(data=glaucoma_stacked_df, aes(x= IOP_Eye, y = IOP_Values, fill = Strain)) +
  geom_boxplot()+ theme_minimal()
```



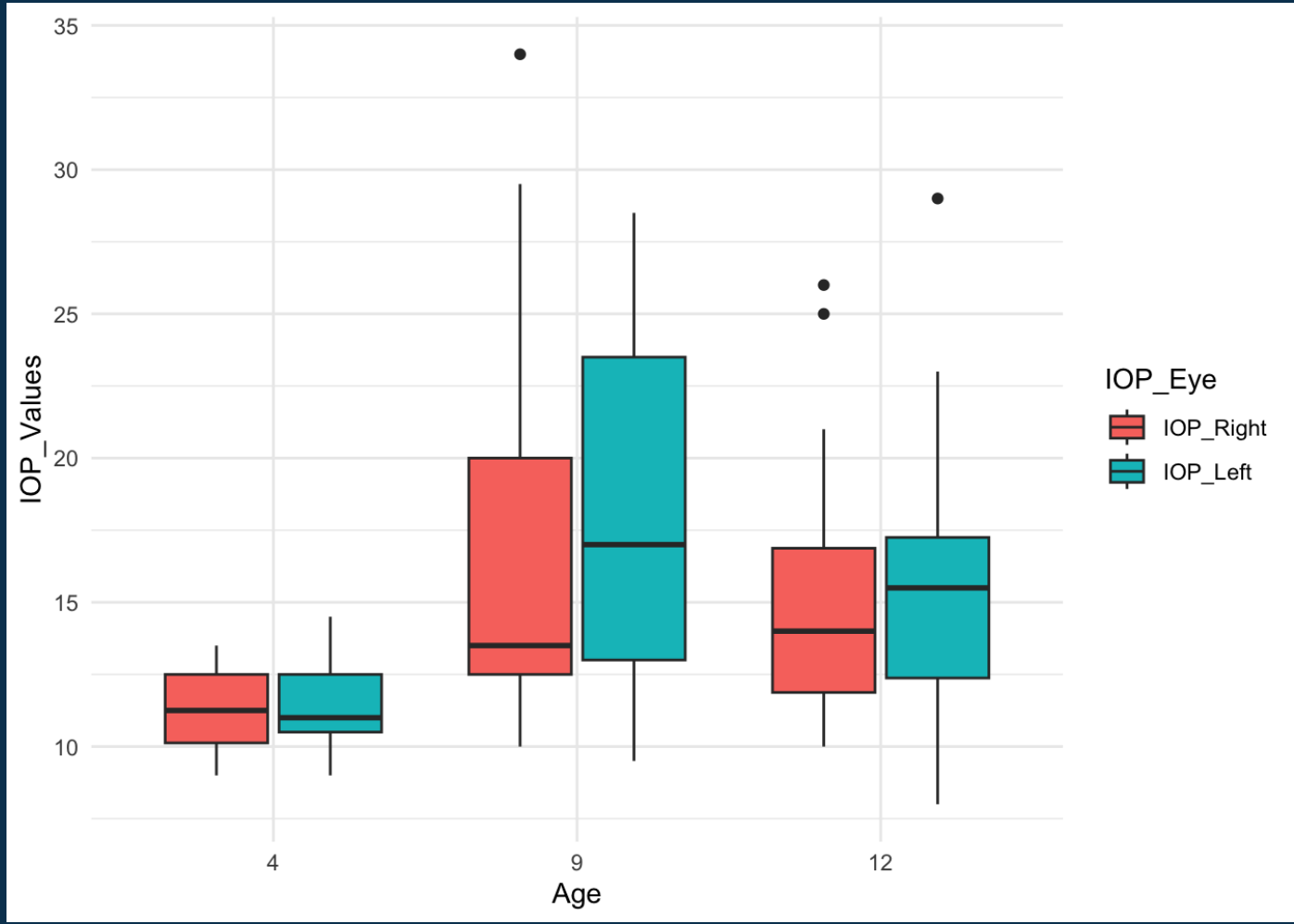
2. **Box plots for IOP values by Strain and by Eye Side**

```
ggplot(data=glaucoma_stacked_df, aes(x= Strain, y = IOP_Values, fill = IOP_Eye)) +
  geom_boxplot()+ theme_minimal()
```



3. **Box plots for IOP values by Age and by Eye Side**

```
ggplot(data=glaucoma_stacked_df, aes(x= as.factor(Age), y = IOP_Values, fill = IOP_Eye  )) +
  geom_boxplot() + xlab('Age')+ theme_minimal()
```

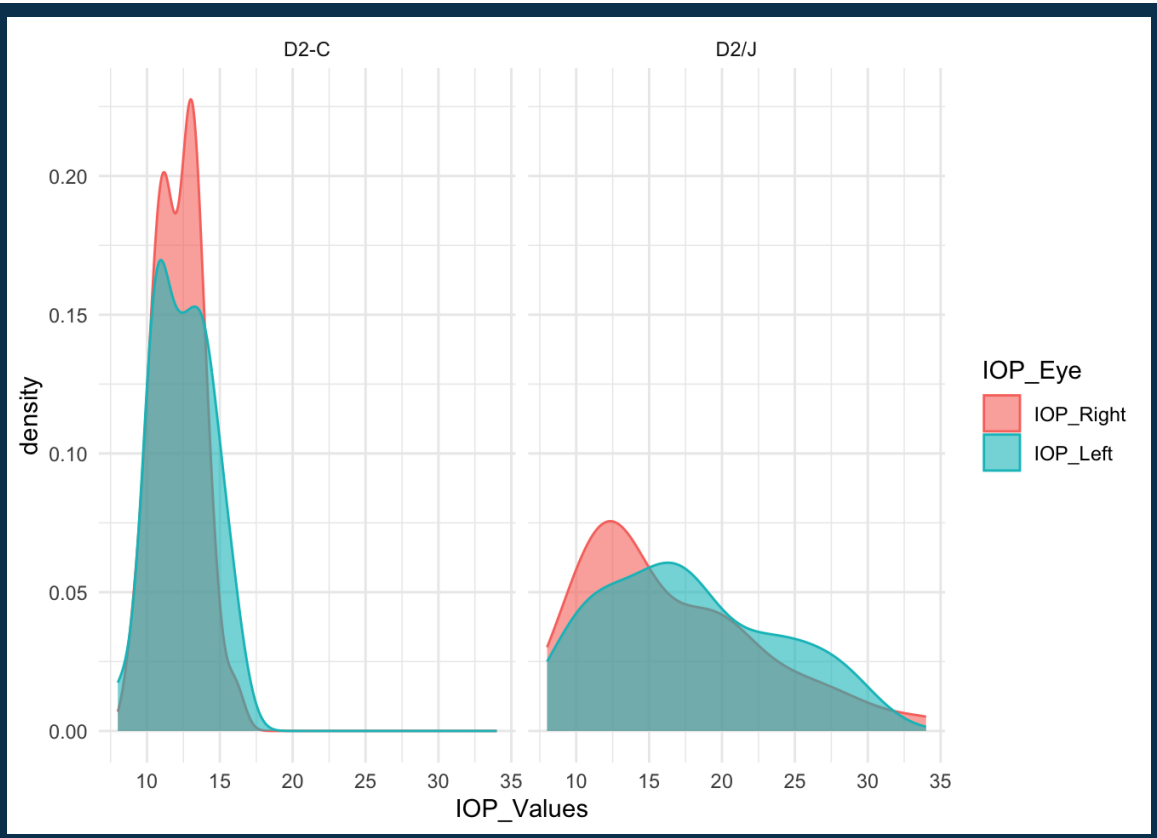


3.4 Density Plots

Density plots help us to understand the data distribution. Here the density plot are drawn by Strain, Age groups.

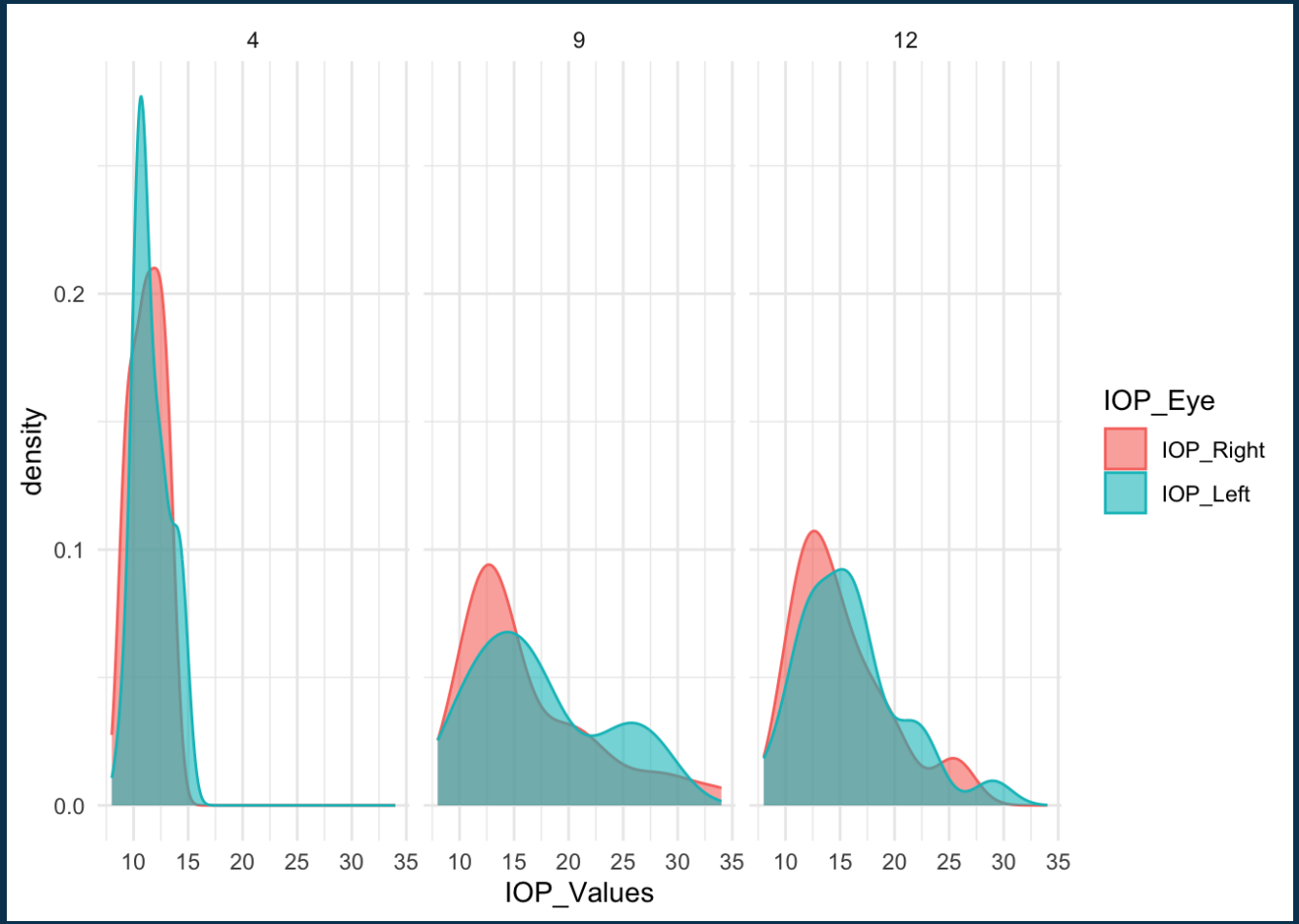
1. **Density plots for IOP values by Strain and by Eye Side**

```
ggplot(glaucoma_stacked_df, aes(x=IOP_Values, fill=IOP_Eye, color=IOP_Eye))+
                        geom_density(alpha=0.6) +
                        facet_wrap(~Strain) +
                         theme_minimal()
```



2. **Density plots for IOP values by Age and by Eye Side**

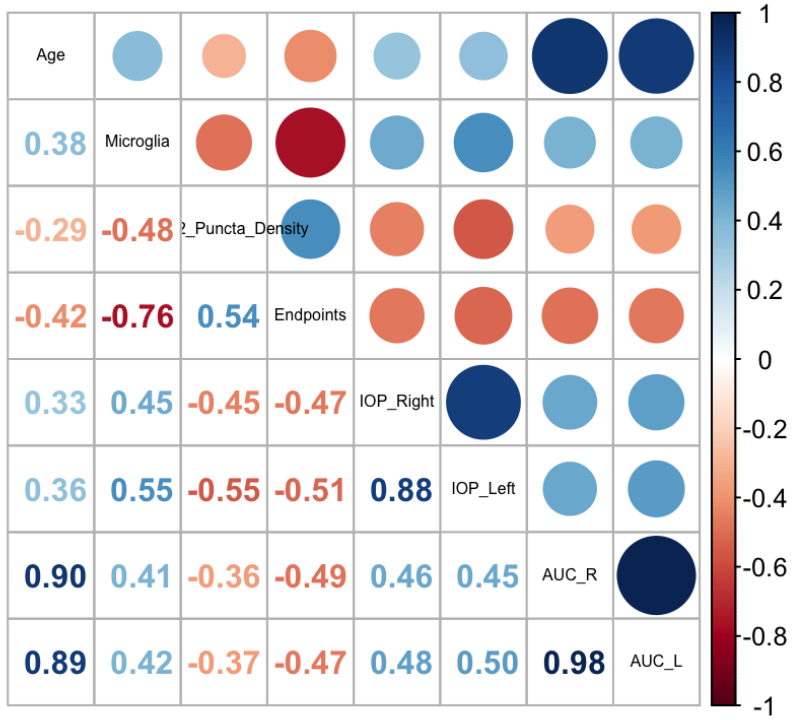
```
ggplot(glaucoma_stacked_df, aes(x=IOP_Values, fill=IOP_Eye, color=IOP_Eye))+
                        geom_density(alpha=0.6) +
                        facet_wrap(~as.factor(Age) ) +
                         theme_minimal()
```



3.5 Correlation Plots

Here a correlation plot is drawn to understand the correlation between the variables in the data set.

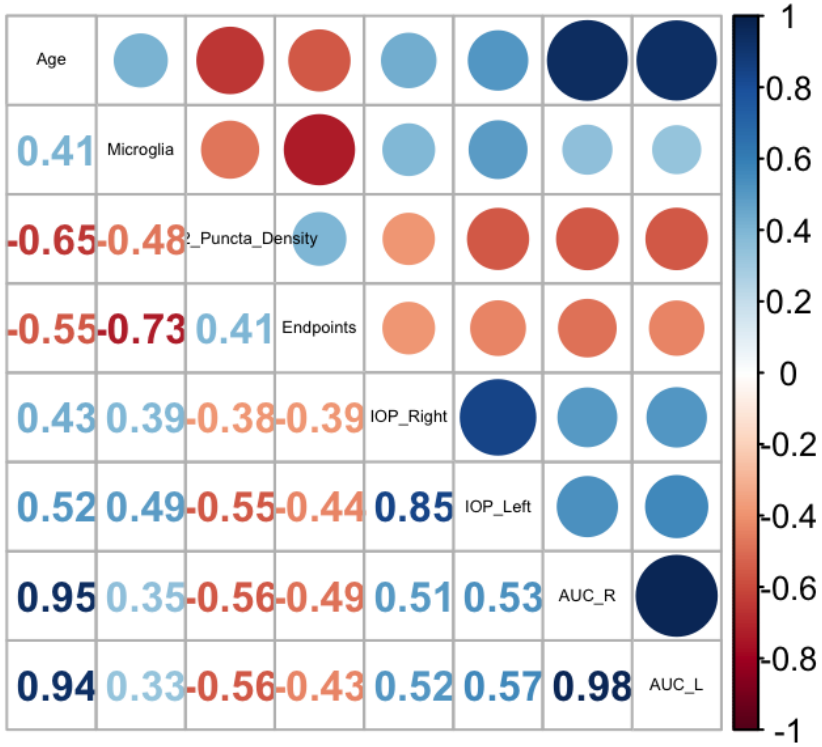
```
library(corrplot)
#Get Correlations
correlation_df = cor(glaucoma_df[,-(2:3)]) # Remove Strain and Sex as they are categorical variables

#Plot Correlations
corrplot.mixed(correlation_df,
               lower = "number",
               upper = "circle",
               tl.col = "black", tl.cex = 0.5
               )
```



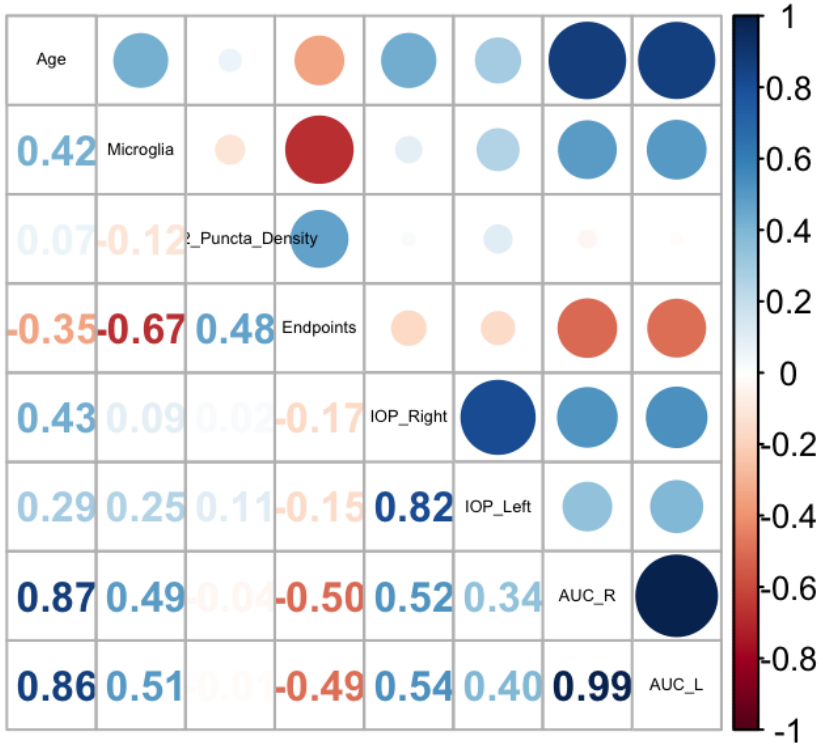
The following correlation plot is is grouped by the `Strain`. (Note: Some data manipulation is done for this graph)

```
library(corrplot)
#Get Correlations
correlation_df = cor(glaucoma_df[,-(2:3)]) # Remove Strain and Sex as they are categorical variables

par(mfrow=c(1,2))

# Plot Correlation for each Strain
for(i1 in unique(glaucoma_df$Strain)) {
            glaucoma_df_i1 = glaucoma_df[glaucoma_df$Strain==i1,]
            correlation_df = cor(glaucoma_df_i1[,-(2:3)])
            corrplot.mixed(correlation_df,
               lower = "number",
               upper = "circle", title = i1,
               tl.col = "black", tl.cex = 0.4,
               mar=c(0,0,2,0)
               )
           }
```

D2/J


Age
0.41 Microglia
-0.65 -0.48 2_Puncta_Density
-0.55 -0.73 0.41 Endpoints
0.43 0.39 -0.38 -0.39 IOP_Right
0.52 0.49 -0.55 -0.44 0.85 IOP_Left
0.95 0.35 -0.56 -0.49 0.51 0.53 AUC_R
0.94 0.33 -0.56 -0.43 0.52 0.57 0.98 AUC_L
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1

D2-C


Age
0.42 Microglia
0.07 -0.12 2_Puncta_Density
-0.35 -0.67 0.48 Endpoints
0.43 0.09 0.02 -0.17 IOP_Right
0.29 0.25 0.11 -0.15 0.82 IOP_Left
0.87 0.49 0.04 -0.50 0.52 0.34 AUC_R
0.86 0.51 0.0 -0.49 0.54 0.40 0.99 AUC_L
1
0.8
0.6
0.4
0.2
0
-0.2
-0.4
-0.6
-0.8
-1

Unsupervised Learning

4 Unsupervised Learning: Clustering

Here we will apply clustering for the **numerical variables** in the data set.

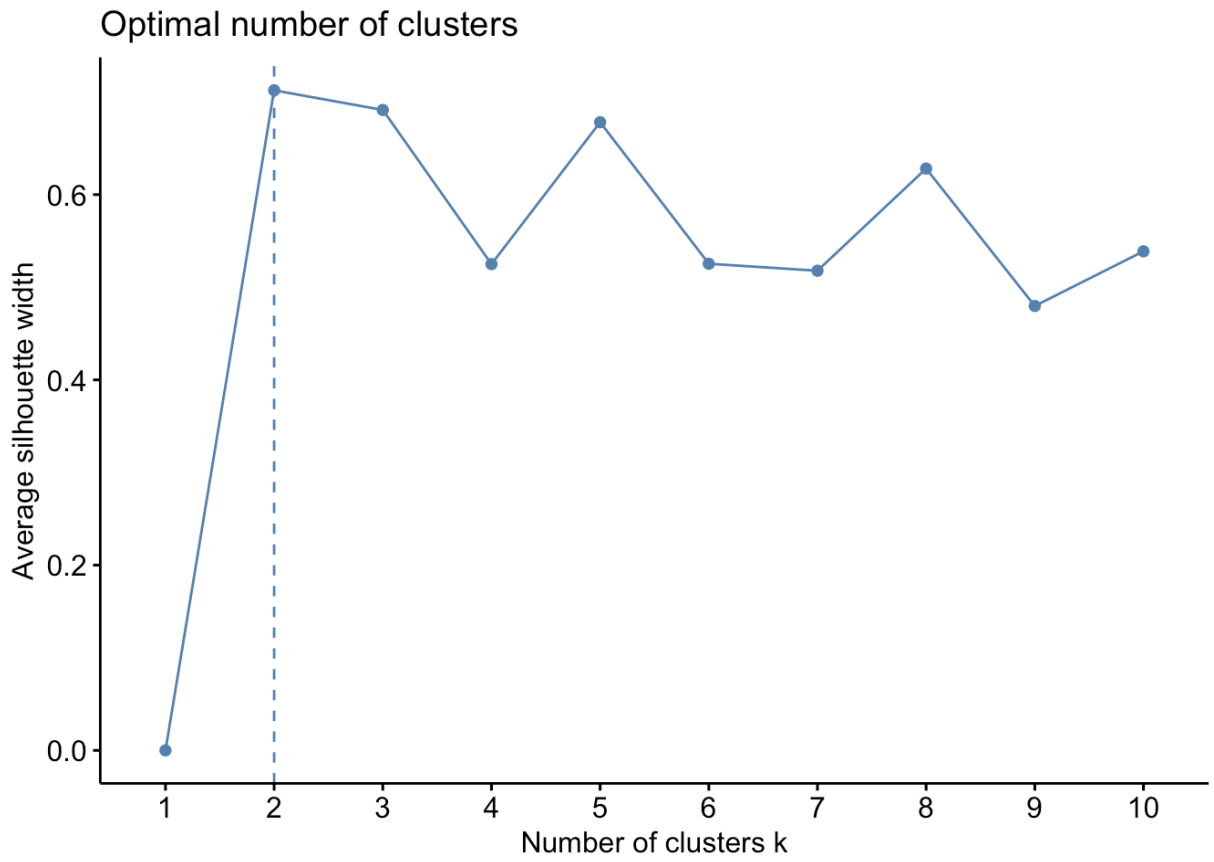
To identify the samples in the data set, we renamed each sample by “Age_Strain_Sex_#”.

```
set.seed(123)
library(factoextra)

glaucoma_df_cluster = glaucoma_df[-c(1:3)]

rownames(glaucoma_df_cluster) = paste(glaucoma_df$Age,'_',glaucoma_df$Strain,'_',glaucoma_df$Sex,'_',ro
wnames(glaucoma_df))
```

4.1 Identify the Number of Required Clusters

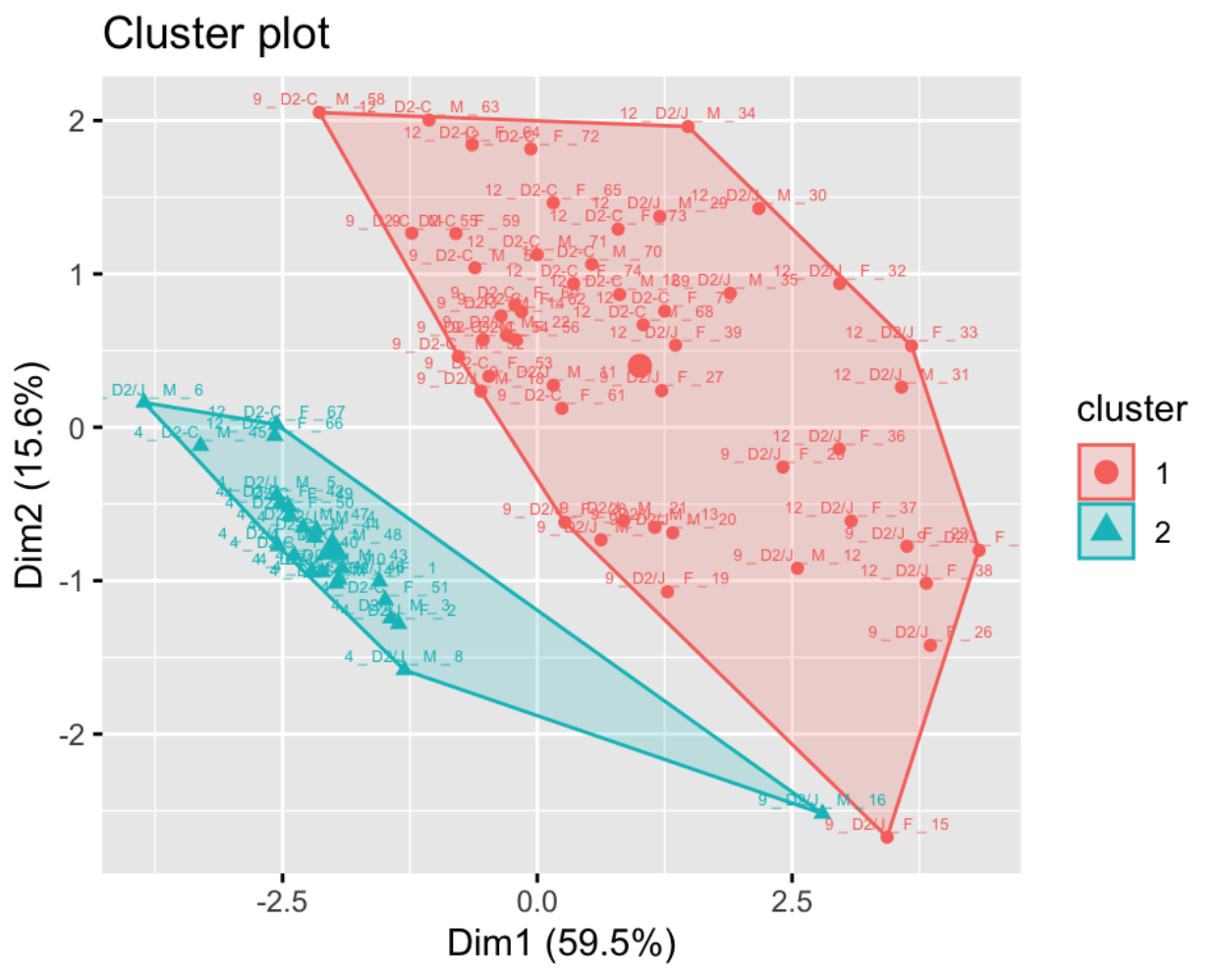
- `Silhouette` method is used to identify the required number of clusters.

```
fviz_nbclust(glaucoma_df_cluster, kmeans, method = "silhouette")
```



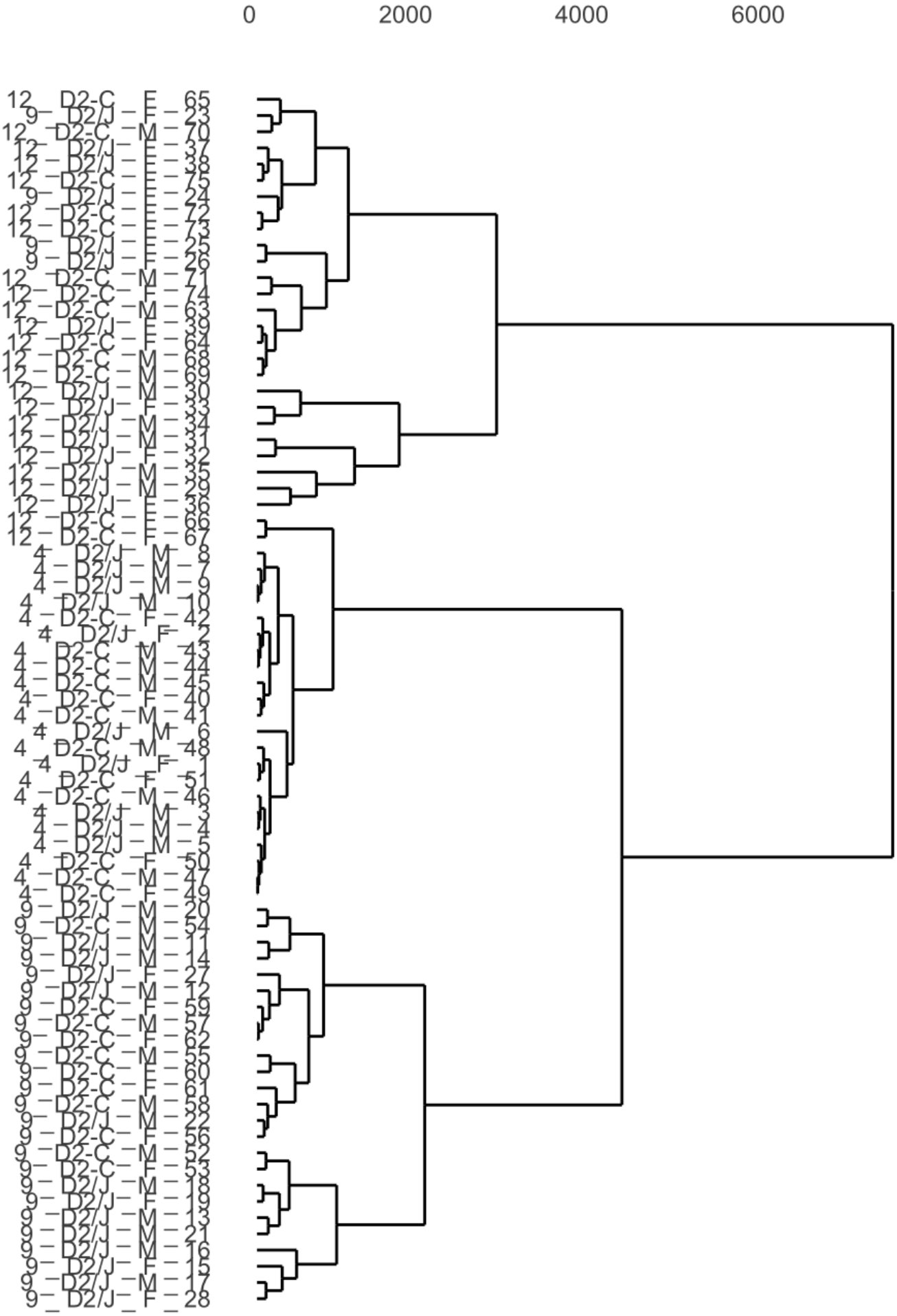
4.2 Apply K-Means Clustering

```
set.seed(123)
k_means_clustering = kmeans(glaucoma_df_cluster, centers = 2)
fviz_cluster(k_means_clustering, data = glaucoma_df_cluster,labelsize = 5)
```



4.3 Apply Hierarchical Clustering

```
library(ggdendro)
set.seed(123)
df_distance = dist(glaucoma_df_cluster, method = 'euclidean')
h_clustering = hclust(df_distance)
ggdendrogram(h_clustering)
```


0
2000
4000
6000
12_D2-C_F_65
9_D2/J_F_23
12_D2-C_M_70
12_D2/J_F_37
12_D2/J_F_38
12_D2-C_F_75
9_D2/J_F_24
12_D2-C_F_72
12_D2-C_F_73
9_D2/J_F_25
9_D2/J_F_26
12_D2-C_M_71
12_D2-C_F_74
12_D2-C_M_63
12_D2/J_F_39
12_D2-C_F_64
12_D2-C_M_68
12_D2-C_M_69
12_D2/J_M_30
12_D2/J_F_33
12_D2/J_M_34
12_D2/J_M_31
12_D2/J_F_32
12_D2/J_M_35
12_D2/J_M_29
12_D2/J_F_36
12_D2-C_F_66
12_D2-C_F_67
4_D2/J_M_8
4_D2/J_M_7
4_D2/J_M_9
4_D2/J_M_10
4_D2-C_F_42
4_D2/J_F_2
4_D2-C_M_43
4_D2-C_M_44
4_D2-C_M_45
4_D2-C_F_40
4_D2-C_M_41
4_D2/J_M_6
4_D2-C_M_48
4_D2/J_F_1
4_D2-C_F_51
4_D2-C_M_46
4_D2/J_M_3
4_D2/J_M_4
4_D2/J_M_5
4_D2-C_F_50
4_D2-C_M_47
4_D2-C_F_49
9_D2/J_M_20
9_D2-C_M_54
9_D2/J_M_11
9_D2/J_M_14
9_D2/J_F_27
9_D2/J_M_12
9_D2-C_F_59
9_D2-C_M_57
9_D2-C_F_62
9_D2-C_M_55
9_D2-C_F_60
9_D2-C_F_61
9_D2-C_M_58
9_D2/J_M_22
9_D2-C_F_56
9_D2-C_M_52
9_D2-C_F_53
9_D2/J_M_18
9_D2/J_F_19
9_D2/J_M_13
9_D2/J_M_21
9_D2/J_M_16
9_D2/J_F_15
9_D2/J_M_17
9_D2/J_F_28

Linear Regression

5.1 Apply Linear Regression Model for Explanation of Dependent Variables

With `lm` function, we can use `anova` function to obtain the effect from each variable.

```
glaucoma_linear_regression = lm(formula = IOP_Values ~ Age + Strain + Sex + IOP_Eye,data = glaucoma_sta
cked_df)
anova(glaucoma_linear_regression)
```

```
## Analysis of Variance Table
##
## Response: IOP_Values
##            Df  Sum Sq Mean Sq F value    Pr(>F)
## Age         1  467.50  467.50 30.4622 1.526e-07 ***
## Strain      1  857.71  857.71 55.8887 6.697e-12 ***
## Sex         1  367.95  367.95 23.9755 2.571e-06 ***
## IOP_Eye     1   19.80   19.80  1.2903    0.2579
## Residuals 145 2225.29   15.35
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

Here we can see that `Age`, `Strain`, and `Sex` are significant variables for the IOP. However, it appears there is no significance difference between `IOP Left Eye` and `IOP Right Eye`.

5.2 Apply Linear Regression Model with Interactions for Explanation of Dependent Variables

With `lm` function, we can use `anova` function to obtain the effect from each variable.

```
glaucoma_linear_regression = lm(formula = IOP_Values ~ (Age + Strain + Sex + IOP_Eye)^2,data = glaucoma
_stacked_df)
anova(glaucoma_linear_regression)
```

```
## Analysis of Variance Table
##
## Response: IOP_Values
##                 Df  Sum Sq Mean Sq F value    Pr(>F)
## Age              1  467.50  467.50 36.3795 1.390e-08 ***
## Strain           1  857.71  857.71 66.7451 1.702e-13 ***
## Sex              1  367.95  367.95 28.6327 3.517e-07 ***
## IOP_Eye          1   19.80   19.80  1.5409 0.2165723
## Age:Strain       1  191.06  191.06 14.8677 0.0001757 ***
## Age:Sex          1    0.89    0.89  0.0696 0.7922956
## Age:IOP_Eye      1    0.95    0.95  0.0737 0.7864051
## Strain:Sex       1  239.16  239.16 18.6105 3.022e-05 ***
## Strain:IOP_Eye   1    6.03    6.03  0.4692 0.4944879
## Sex:IOP_Eye      1    0.97    0.97  0.0755 0.7838605
## Residuals      139 1786.23   12.85
## ---
## Signif. codes:  0 '***' 0.001 '**' 0.01 '*' 0.05 '.' 0.1 ' ' 1
```

The interactions `Age:Strain` and `Strain:Sex` are significant

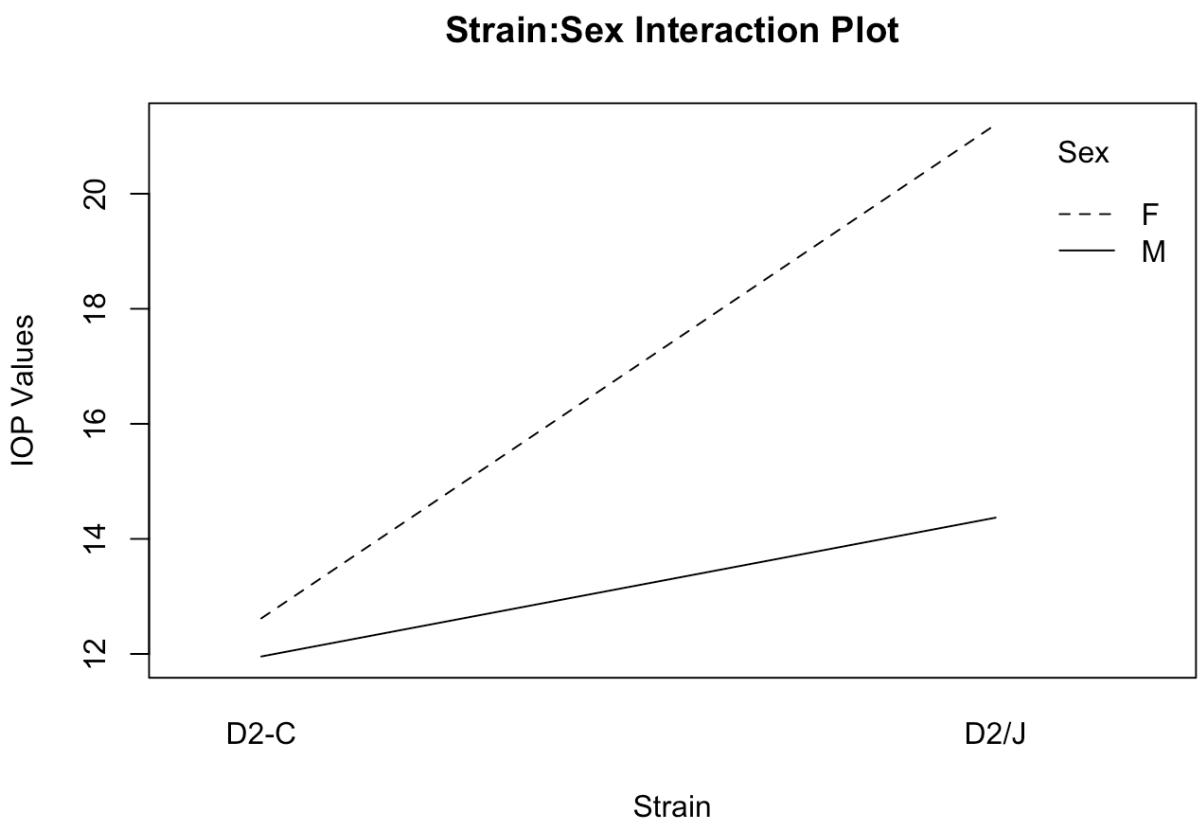
- For different Age levels, the average IOP values are significantly different between Strains
- When Sex is change from M to F, the average IOP values are significantly different between Strains

```
interaction.plot(x.factor = glaucoma_stacked_df$Age,
                 trace.factor = glaucoma_stacked_df$Sex,
                 response = glaucoma_stacked_df$IOP_Values,
                 fun = mean,
                 xlab="Age",
                 ylab="IOP Values", trace.label="Sex" ,
                 main = "Age:Sex Interaction Plot")
```

The difference of IOP values between Female and Male significantly increase when the Age change from 4 to 9

The difference of IOP values between Female and Male significantly decrease when the Age change from 9 to 12

```
interaction.plot(x.factor = glaucoma_stacked_df$Strain,
                 trace.factor = glaucoma_stacked_df$Sex,
                 response = glaucoma_stacked_df$IOP_Values,
                 fun = mean,
                 xlab="Strain",
                 ylab="IOP Values",
                 trace.label="Sex" ,
                 main = "Strain:Sex Interaction Plot")
```



The difference of IOP values between Female and Male significantly higher for control to treatment

Predictive Modeling

Supervised Learning

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis
- Descriptive Statistics
- Plot the data

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

5.3 Create Predictor Models

- Response Variable: `IOP Values`
- Dependent Variables: `Age` , `Strain` , `Sex` , `IOP_Eye`

5.3.1 Train-Test Split

Apply train-test split with 70% as `train data set` and 30% as `test data set`.

- `glaucoma_stacked_df_train` - Training Data Set
- `glaucoma_stacked_df_test` - Testing Data Set

```
set.seed(1234)
sample <- sample(c(TRUE, FALSE), nrow(glaucoma_stacked_df), replace=TRUE, prob=c(0.7,0.3))
glaucoma_stacked_df_train  <- glaucoma_stacked_df[sample, ]
glaucoma_stacked_df_test   <- glaucoma_stacked_df[!sample, ]
```

Supervised Learning

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis
- Descriptive Statistics
- Plot the data

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

5.3.2 Train the Linear Regression Model with Cross Validation

- 4 Folds are applied for cross validation (This can be changed)

```r
library(caret)

cross_validate_setup = trainControl(method = "cv", number = 4) # Cross validation with 4 folds

linear_regression_cv = train(IOP_Values ~ (Age + Strain + Sex + IOP_Eye)^2,
                             data = glaucoma_stacked_df_train,
                             trControl = cross_validate_setup,
                             method = "lm",
                             metric = 'RMSE'       # RMSE is used to compare the model
                       )

linear_regression_cv
```

```
## Linear Regression
##
## 112 samples
##   4 predictor
##
## No pre-processing
## Resampling: Cross-Validated (4 fold)
## Summary of sample sizes: 84, 85, 83, 84
## Resampling results:
##
##   RMSE      Rsquared   MAE
##   3.907711  0.4828074  2.933547
##
## Tuning parameter 'intercept' was held constant at a value of TRUE
```

5.3.3 Train the Random Forest Regression Model with Cross Validation

```
cross_validate_setup = trainControl(method = "cv", number = 4) # Cross validation with 4 folds

random_forest_regression_cv = train(IOP_Values ~ (Age + Strain + Sex + IOP_Eye),
                              data = glaucoma_stacked_df_train,
                              trControl = cross_validate_setup,
                              method = "rf",
                              metric = 'RMSE'       # RMSE is used to compare the model
                      )

random_forest_regression_cv
```

```
## Random Forest
##
## 112 samples
##   4 predictor
##
## No pre-processing
## Resampling: Cross-Validated (4 fold)
## Summary of sample sizes: 83, 85, 85, 83
## Resampling results across tuning parameters:
##
##   mtry  RMSE      Rsquared   MAE
##   2     3.644183  0.5517190  2.660361
##   3     3.714248  0.5396859  2.675851
##   4     3.745871  0.5341173  2.655340
##
## RMSE was used to select the optimal model using the smallest value.
## The final value used for the model was mtry = 2.
```

5.3.4 Train the Boosting Model with Cross Validation

```
cross_validate_setup = trainControl(method = "cv", number = 4) # Cross validation with 4 folds

boosting_regression_cv = train(IOP_Values ~ (Age + Strain + Sex + IOP_Eye),
                               data = glaucoma_stacked_df_train,
                               trControl = cross_validate_setup,
                               method = "gbm",
                               metric = 'RMSE'       # RMSE is used to compare the model
                       )
```

```
boosting_regression_cv
```

```
## Stochastic Gradient Boosting
##
## 112 samples
##   4 predictor
##
## No pre-processing
## Resampling: Cross-Validated (4 fold)
## Summary of sample sizes: 85, 83, 84, 84
## Resampling results across tuning parameters:
##
##   interaction.depth  n.trees  RMSE      Rsquared   MAE
##   1                   50      3.965290  0.4761767  3.142700
##   1                  100      3.988170  0.4657997  3.223047
##   1                  150      3.976582  0.4663227  3.247917
##   2                   50      3.678288  0.5442205  2.937053
##   2                  100      3.615583  0.5580590  2.895679
##   2                  150      3.636920  0.5539942  2.899615
##   3                   50      3.654926  0.5465249  2.891683
##   3                  100      3.618513  0.5596161  2.909455
##   3                  150      3.627240  0.5571990  2.892020
##
## Tuning parameter 'shrinkage' was held constant at a value of 0.1
## Tuning parameter
##  'n.minobsinnode' was held constant at a value of 10
## RMSE was used to select the optimal model using the smallest value.
## The final values used for the model were n.trees = 100, interaction.depth = 2, shrinkage = 0.1
##  and n.minobsinnode = 10.
```

Supervised Learning

Research Problem

Data Collection
- Collect data by sampling or design of experiment

Data Cleaning and Preprocessing
- Data inspection
- Identify outliers, and missing values

Exploratory Data Analysis
- Descriptive Statistics
- Plot the data

Split Data Set
- Training Data Set (~70%)
- Test Data Set (~30%)

Model Development: Apply Predictive Models to Training Data
- e.g., linear regression, random forest, boosting methods

Model Training and Tuning: Apply Cross Validation for Training Data
- Evaluate Hyperparameters
- Control Overfitting

Model Evaluation: Apply the prediction model in the Test Data set

5.3.5 Apply `test data set` to compare the performance of two models

```
linear_regression_prediction = predict(linear_regression_cv,newdata = glaucoma_stacked_df_test)
random_forest_prediction     = predict(random_forest_regression_cv,newdata = glaucoma_stacked_df_test)
boosting_prediction          = predict(boosting_regression_cv,newdata = glaucoma_stacked_df_test)

linear_regression_rmse_test = RMSE(pred = linear_regression_prediction, obs = glaucoma_stacked_df_test
$IOP_Values)

random_forest_rmse_test     = RMSE(pred = random_forest_prediction, obs = glaucoma_stacked_df_test$IOP_
Values)

boosting_rmse_test          = RMSE(pred = boosting_prediction, obs = glaucoma_stacked_df_test$IOP_Value
s)


print(paste('RMSE of Test Data: Linear Regression = ', linear_regression_rmse_test))
```

```
## [1] "RMSE of Test Data: Linear Regression =  3.32033784086963"
```

```
print(paste('RMSE of Test Data: Random Forest Regression = ', random_forest_rmse_test))
```

```
## [1] "RMSE of Test Data: Random Forest Regression =  3.09650465557282"
```

```
print(paste('RMSE of Test Data: Boosting Regression = ', boosting_rmse_test))
```

```
## [1] "RMSE of Test Data: Boosting Regression =  3.10053564256255"
```

`Random Forest Regression` model has the lowest test RMSE; therefore, we can use that for prediction purpose.